Industrial Mini-SOC: Detection and Response for Simulated IT/OT Cybersecurity Incidents

Independent cybersecurity engineering technical report

Author: Davood Noori

Abstract

This report documents Industrial Mini-SOC, a small cybersecurity monitoring prototype for selected IT/OT-style events. The project generates simulated industrial telemetry, reads synthetic authentication and network logs, compares observed devices with an approved inventory, checks outbound connections against an allowlist, writes alerts as JSONL evidence, and exports selected alerts into a Wazuh-ready format. Five incident notes are included: temperature spike, failed login burst, port scan, unknown device, and suspicious outbound connection.

The work was carried out as a self-directed technical project during personal study time. The goal was not to imitate a full production SOC, but to build a small system where the whole path can be inspected: input data, detection logic, alert evidence, incident notes, SIEM validation, and final reporting. The evaluation uses benign simulator baselines, repeated anomaly runs, timed sample-log detector runs, automated tests, and three Wazuh Docker validation paths. The results are consistent inside the controlled data set. The limits are equally important: the telemetry is synthetic, the rules are narrow, and the Wazuh work validates selected paths rather than a complete monitoring environment.

1. Introduction

Small industrial environments often combine ordinary IT systems with gateways, sensors, operator workstations, and process-oriented devices. Even in a lab, this kind of environment becomes difficult to reason about unless the evidence is collected and written down in a structured way. Industrial Mini-SOC was built to practise that workflow at a scale that can be handled by one person without hiding the details behind a large platform.

The first implementation stays intentionally small. It proves the chain from input data to detection, alert evidence, incident note, and report evidence before adding more infrastructure. The latest phase extends the same chain into Wazuh for three controlled validation paths: an exported Mini-SOC alert, a Docker-contained syslog-style SSH authentication event, and a failed-SSH syslog event sent from a separate endpoint container. A persistent Wazuh-agent endpoint, hardware telemetry, and broader network monitoring are left for later work because they require a separate lab setup.

2. Aim and Research Questions

The aim is to design, implement, and evaluate a low-cost lab monitoring prototype that can make selected IT/OT-style security events visible and explainable.

Research questions:

1. Which simulated IT/OT security events can be detected using a low-cost Mini-SOC architecture?

2. How accurately can the prototype detect selected incidents such as abnormal sensor values and suspicious control commands?
3. How clearly can the prototype explain alerts for a junior analyst or non-specialist technical user?
4. What are the main limitations of using simulated data for this kind of monitoring work?

3. Background

A Security Operations Centre, or SOC, monitors events, investigates alerts, and coordinates incident response. A Security Information and Event Management system, or SIEM, supports this work by collecting logs, applying detection rules, and presenting alerts to analysts.

IT/OT security is concerned with the interaction between traditional information technology and operational technology such as industrial devices, controllers, sensors, and process systems. This project does not interact with real industrial equipment. Instead, it uses safe, simulated OT-like events such as abnormal sensor values, unexpected commands, and invalid machine states.

Detection engineering is the process of defining suspicious conditions, implementing detection logic, validating results, and tuning rules to reduce false positives and missed detections. In this project, the detection engineering work is first implemented in Python and then partly validated through Wazuh to check that selected alert paths can also be handled by a real SIEM.

4. Threat Model

The lab models a small controlled environment with a simulated industrial device, sample host and network logs, alert logic, SIEM export, and analyst documentation.

Key assets include:

Asset	Description	Security relevance
Industrial simulator	Python component that emits sensor and command events	Source of normal and suspicious process telemetry
Event logs	JSONL files containing generated events	Primary evidence for detection and analysis
Detection engine	Python logic that identifies abnormal events	Produces analyst-facing alerts
Alert logs	JSONL files containing detection results	Evidence for investigation and reporting
Incident reports	Markdown reports generated from scenario output	Supports documentation and lessons learned
Wazuh export path	JSONL bridge from internal alerts to Wazuh ingestion	Tests whether selected local alerts can be validated in a SIEM
Report artefacts	Markdown and DOCX project reports	Preserve design decisions, results, and limitations

The implemented prototype covers abnormal sensor values, unexpected commands, invalid machine states, failed SSH login bursts, port-scan-like connection behaviour, unknown-device observations, and suspicious outbound connections.

5. System Design

The local detection workflow uses a simple evidence pipeline:

```
Python simulator -> JSONL event log -> Python detector -> JSONL alert log -> CSV results table -> incident report
```

The Wazuh-facing extension adds controlled SIEM validation steps:

```
Python alert JSONL -> Wazuh export JSONL -> Wazuh localfile ingestion -> Wazuh rule match -> Wazuh indexer evidence
Syslog-style auth line -> Wazuh localfile ingestion -> Wazuh sshd decoder -> Wazuh rule match -> dashboard evidence
Endpoint container -> UDP syslog -> Wazuh sshd decoder -> Wazuh rule match -> dashboard evidence
```

The implementation is organised as a Python package under `src/industrial_mini_soc/`. Each detector is kept independent so it can be tested and later connected to another collection or SIEM workflow without changing the earlier scenarios.

Module	Purpose
`events.py`	Shared event and alert dataclasses, JSONL helpers, timestamp utilities
`simulator.py`	Generates normal and anomalous industrial telemetry
`detect.py`	Applies detection logic to event records
`runner.py`	Runs a complete scenario and records evidence
`batch.py`	Runs repeated simulator evaluations and writes the prototype evaluation summary
`log_evaluation.py`	Evaluates sample-log detectors and records local processing time
`reporting.py`	Produces Markdown incident reports
`auth_logs.py`	Parses Linux auth logs and detects failed-login bursts
`network_logs.py`	Parses connection-attempt logs and detects port-scan-like behaviour
`inventory.py`	Compares observed devices against an approved asset inventory
`outbound.py`	Compares outbound connections against an approved destination allowlist
`wazuh_export.py`	Converts internal alert JSONL records into a flat JSON format for Wazuh ingestion
`wazuh_preflight.py`	Checks the exported alert format against the local Wazuh rules and localfile example

A separate coverage matrix in `docs/detection-matrix.md` maps each detection to its input source, evidence, Wazuh mapping, and validation state.

The Python prototype avoids external runtime dependencies, which keeps the local detectors easy to run and the evidence files readable without special tooling. Docker is used only for Wazuh validation. The Wazuh files have been tested in the Docker single-node lab for one exported suspicious outbound alert, one Docker-contained SSH authentication event, and one endpoint-container syslog event.

6. Implementation

The simulator currently supports six scenarios:

Scenario	Description
`normal`	Generates baseline telemetry without alerts
`noisy-normal`	Generates benign but less tidy baseline telemetry without alerts
`temp-spike`	Inserts one abnormal temperature event
`vibration-spike`	Inserts one abnormal vibration event
`bad-command`	Inserts one unexpected control-style command
`impossible-state`	Inserts one invalid machine state

Events are written as one JSON object per line. This format is suitable for later ingestion by log-processing tools and is straightforward to inspect during development.

The detector set currently includes:

- temperature spike detection
- vibration spike detection
- unexpected OT-style command detection
- impossible machine state detection
- failed SSH login burst detection
- port-scan-like connection behaviour detection
- unknown-device detection against an asset inventory
- suspicious outbound connection detection against an allowlist

The scenario runner connects the simulator and detector into one repeatable workflow. It writes event evidence, alert evidence, a row in `data/results.csv`, and an optional incident report.

The authentication-log detector uses synthetic Linux `sshd` log samples. It raises a medium-severity alert when at least five failed SSH password attempts for the same source IP, username, and host occur within a 120-second window. The same sample family was later used for Wazuh validation through Wazuh's native `sshd` decoder.

The network-log detector uses synthetic connection-attempt samples. It raises a medium-severity alert when one source IP contacts at least six unique destination ports on the same target IP within a 60-second window. Repeated attempts to the same port do not satisfy the rule.

The inventory detector uses a synthetic approved asset inventory and observed-device log. It raises a medium-severity alert when an observed IP/MAC combination does not match the approved inventory baseline.

The outbound detector uses a synthetic destination allowlist and outbound-connection log. It raises a medium-severity alert when a destination, port, and protocol combination is not approved.

The Wazuh exporter converts Mini-SOC alert records into flat JSONL entries with fields such as `integration`, `event_kind`, `detection_name`, `severity`, `wazuh_level`, `source`, `target`, and

`evidence`. The flat field layout is intentional because the first Wazuh rules can match those values directly. A local preflight command checks the exported sample alert against the Wazuh rules before live testing. The Docker helper starts the Wazuh single-node stack, installs the local rules, runs `wazuh-logtest`, injects a fresh localfile event, and verifies that rule `110104` appears in both the manager alert log and the indexer.

7. Experimental Method

The prototype is evaluated by running controlled samples through the same basic path: parse input, apply a detection rule, write alert evidence, and document the outcome.

The temperature-spike scenario can be reproduced with this command pattern:

```
PYTHONPATH=src python3 -m industrial_mini_soc.runner \
  --scenario temp-spike \
  --count 20 \
  --run-id demo-001 \
  --incident-report incidents/INC-001-temperature-spike.md
```

The run generates:

Evidence	Location
Event log	`data/runs/demo-001-temp-spike-events.jsonl`
Alert log	`data/runs/demo-001-temp-spike-alerts.jsonl`
Results table	`data/results.csv`
Incident report	`incidents/INC-001-temperature-spike.md`

The repeated simulator evaluation was run with:

```
PYTHONPATH=src python3 -m industrial_mini_soc.batch \
  --batch-id prototype-eval-001 \
  --repeats 3 \
  --count 20 \
  --summary report/prototype-evaluation-summary.md
```

The sample-log detector evaluation was run with:

```
PYTHONPATH=src python3 -m industrial_mini_soc.log_evaluation \
  --summary report/log-detector-evaluation-summary.md
```

Automated tests verify parser behaviour, detection behaviour, CLI execution, report-supporting workflows, timing-field recording, sample-log evaluation, Wazuh export mapping, Wazuh preflight validation, and negative cases where normal samples should not alert.

8. Evaluation Results

The simplest local case is the temperature-spike scenario. It generated 20 simulator events and one alert.

Scenario	Run ID	Event count	Alert count	Detected	First detection	Highest severity
`temp-spike`	`demo-001`	20	1	yes	Temperature spike	medium

The alert explanation stated that the temperature exceeded the lab threshold for normal process behaviour. The incident write-up classified the result as a true positive for a controlled scenario.

The repeated simulator evaluation used batch `prototype-eval-001`. Each of the six simulator scenarios was executed three times, producing 18 total runs and 360 generated events. The `normal` and `noisy-normal` baselines produced no alerts. Each anomaly scenario produced one alert per run.

Scenario	Runs	Detected runs	Total alerts	Outcome	First detection	Highest severity
`normal`	3	0	0	Baseline stayed quiet	`none`	none
`noisy-normal`	3	0	0	Noisy baseline stayed quiet	`none`	none
`temp-spike`	3	3	3	All controlled anomaly runs alerted	Temperature spike	medium
`vibration-spike`	3	3	3	All controlled anomaly runs alerted	Vibration spike	medium
`bad-command`	3	3	3	All controlled anomaly runs alerted	Unexpected command	high
`impossible-state`	3	3	3	All controlled anomaly runs alerted	Invalid machine state	medium

The summary also records timing fields for the simulator workflow. Event-to-alert latency is measured from the timestamp of the triggering simulator event to the timestamp written on the first alert. Detector processing time is measured around the local detection function. These values describe local batch-file processing, not live SIEM latency.

The timing observations from `prototype-eval-001` were:

- Baseline event-to-alert latency was not applicable because the `normal` and `noisy-normal` runs generated no alerts.
- Anomaly event-to-alert latency was between `0.000` and `0.001` seconds in the recorded batch.
- Average detector processing time by simulator scenario was between `0.049` and `2.882` ms.

The batch summary is stored in `report/prototype-evaluation-summary.md`. Raw event and alert evidence is generated under `data/` and excluded from version control to keep the repository focused on source files and reproducible samples.

The sample-log evaluation is stored in `report/log-detector-evaluation-summary.md`. It contains nine sample cases across authentication, network, inventory, and egress monitoring. Four cases produced alerts, five cases stayed quiet, and all nine cases matched the expected outcome. The evaluation records local parse-and-detect processing time, not live Wazuh ingestion delay.

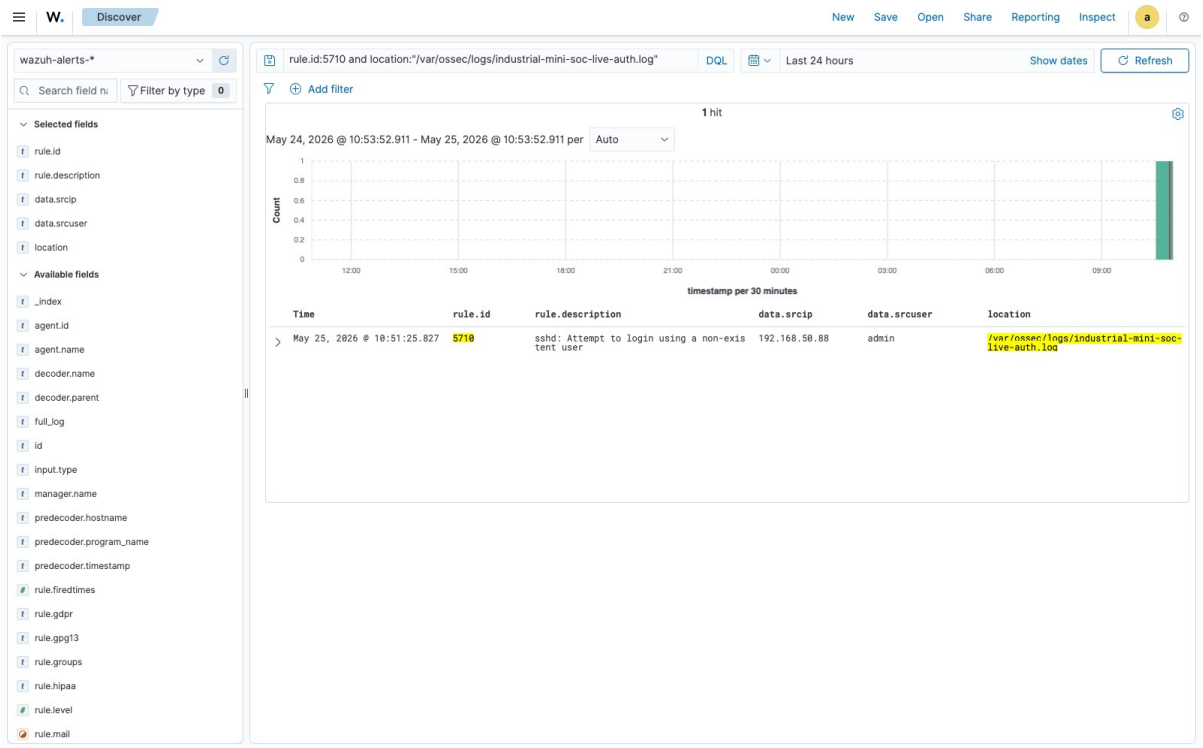
Evaluation group	Sample cases	Alerts	Quiet cases	Local processing time
Log detector samples	9	4	5	Between `0.094` and `9.736` ms in the recorded run

The automated test suite passes with 41 tests. The tests cover normal and noisy event generation, anomaly generation, expected detections, negative samples, CLI execution, runner output, results recording, incident report creation, repeated batch execution, batch summary generation, log-detector evaluation, Wazuh export mapping, Wazuh preflight validation, and basic script validation.

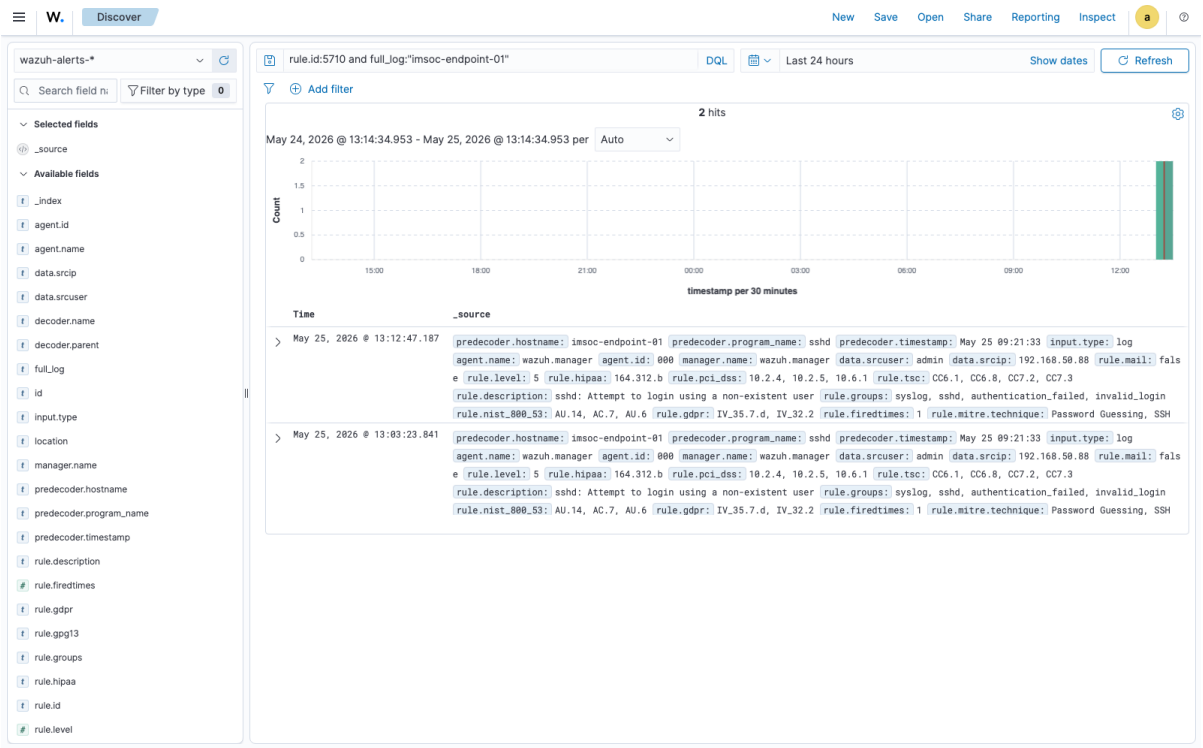
For authentication monitoring, the project uses a small Linux `sshd` sample. The sample contains five failed SSH password attempts for the user `admin` from `192.168.50.88` between 19:12:41 and 19:13:33. The detector parsed five failed-login events and generated one medium-severity `failed_login_burst` alert.

Scenario	Log source	Parsed events	Alert count	Detection	Severity
failed-login-burst	`data/samples/auth.log`	5	1	Failed login burst	medium

The incident notes for this case are kept in `incidents/INC-002-failed-login-burst.md`. The same safe SSH-style event family was also tested through Wazuh in two ways: first through a Wazuh-read local authentication log, and then through a short-lived endpoint container that sent syslog over UDP/514. In both cases Wazuh used its native `sshd` decoder and matched rule `5710`.



Wazuh Discover view filtered to rule 5710 for the live auth-log validation



Wazuh Discover view filtered to rule 5710 for the endpoint syslog validation

For reconnaissance-style behaviour, the project uses a connection-attempt sample. One source, `192.168.50.90`, contacts seven unique ports on `192.168.50.20` within 24 seconds. The detector parsed

ten connection attempts and generated one medium-severity `port_scan` alert. Separate normal and repeated-single-port samples generated zero alerts.

Scenario	Log source	Parsed attempts	Alert count	Detection	Severity
port-scan	`data/samples/connections.log`	10	1	Port scan	medium

The incident notes for this case are kept in `incidents/INC-003-port-scan.md`.

For inventory drift, the project compares an approved asset list with observed devices. The sample contains one unapproved device, `192.168.50.77` with MAC address `de:ad:be:ef:00:77`. The detector loaded five approved inventory assets, parsed five observed devices, and generated one medium-severity `unknown_device` alert. A known-devices sample generated zero alerts.

Scenario	Baseline	Observed devices	Alert count	Detection	Severity
unknown-device	`data/samples/assets.csv`	5	1	Unknown device	medium

The incident notes for this case are kept in `incidents/INC-004-unknown-device.md`.

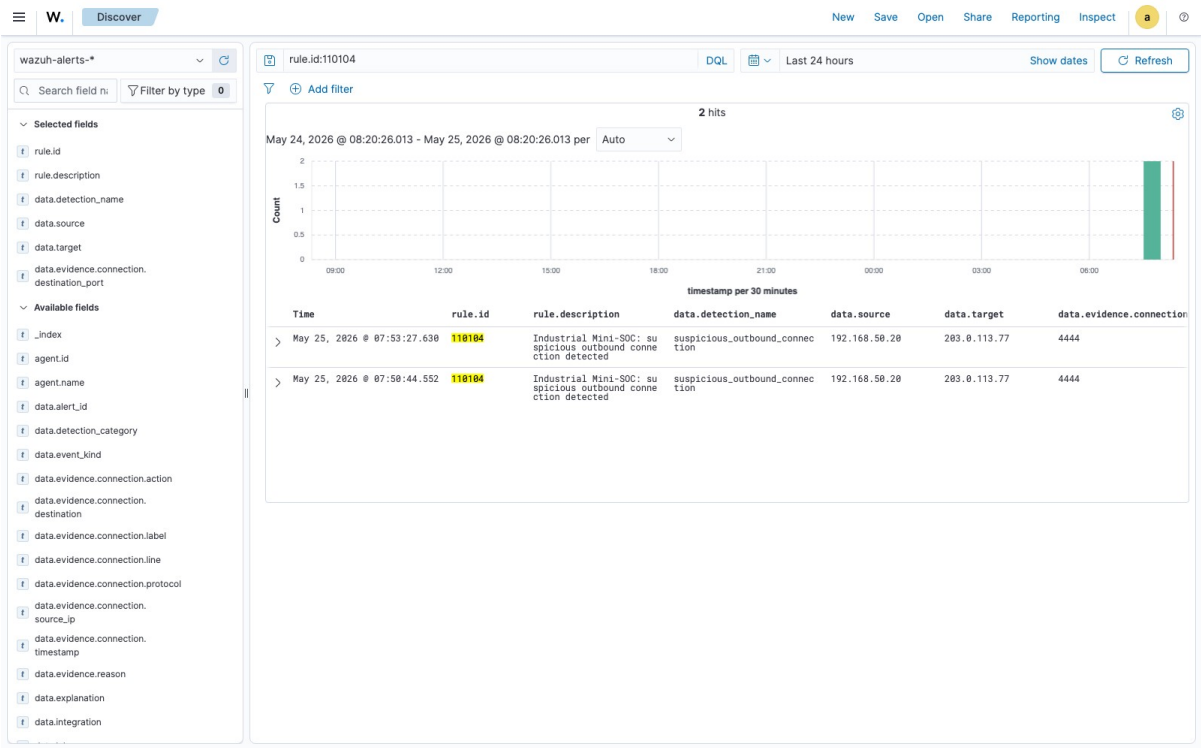
For egress monitoring, the project compares outbound connections with an approved destination allowlist. The sample contains one unapproved connection from `192.168.50.20` to `203.0.113.77` on TCP port `4444`. The detector loaded four approved destination entries, parsed four outbound connections, and generated one medium-severity `suspicious_outbound_connection` alert. The normal outbound sample generated zero alerts.

Scenario	Baseline	Parsed connections	Alert count	Detection	Severity
suspicious-outbound-connection	`data/samples/allowed-destinations.csv`	4	1	Suspicious outbound connection	medium

The incident notes for this case are kept in `incidents/INC-005-suspicious-outbound.md`.

The Wazuh export and validation path converts an internal alert JSONL file into `mini_soc_alert` JSON records for localfile ingestion. Unit tests confirm the field mapping, severity-to-Wazuh-level mapping, CLI round trip, local preflight checks, and basic script checks. The Wazuh configuration and local rule files have been tested in a Docker single-node Wazuh lab at version `4.14.5`.

The first live Wazuh validation used the suspicious outbound sample. The exported alert was decoded as JSON, matched local rule `110104`, written to `/var/ossec/logs/alerts/alerts.json`, returned from the Wazuh indexer, and displayed in the Wazuh dashboard Discover view. The second validation used a Docker-contained syslog-style failed-SSH line. The third validation used a separate endpoint container that sent a failed-SSH syslog event to the Wazuh manager over UDP/514. Wazuh decoded both authentication events with the native `sshd` decoder and matched built-in rule `5710`.



Wazuh Discover view filtered to rule 110104 for the suspicious outbound connection alert

The supporting Wazuh artefacts are kept in the repository instead of being copied into the report as a long appendix. The export and preflight code is in `src/industrial_mini_soc/`, the Wazuh configuration examples are in `config/wazuh/`, the local rules are in `rules/wazuh/local_rules.xml`, the helper scripts are in `scripts/`, and the validation evidence is split between `evidence/wazuh/` and `screenshots/`. That keeps the report readable while still making the evidence traceable.

9. Research Question Findings

The first research question was about coverage. In its current form, the project detects process anomalies, suspicious control behaviour, invalid machine states, failed SSH login bursts, port-scan-like connection attempts, unknown devices, and unapproved outbound connections. The coverage is deliberately narrow, but it is broad enough to exercise several common monitoring inputs: simulator telemetry, host-style authentication logs, connection logs, inventory observations, and an egress allowlist.

The second research question was about accuracy on the selected incidents. In the controlled evaluation set, the result is consistent: the two benign simulator baselines produced no alerts, all four simulator anomaly scenarios alerted in every repeated run, and all nine sample-log cases matched their expected outcomes. This is controlled test-set accuracy. It should not be read as production accuracy, because the rules have not yet been exposed to long-running endpoint logs, real network noise, malformed records, or deliberate evasion.

The third research question was about explanation quality. The alerts contain a detection name, severity, source or target context where relevant, evidence text, and a recommended response. The incident notes preserve the input and alert paths and turn each result into a short analyst-facing record. That is enough

for a junior analyst to follow the lab cases, but it is still far simpler than a real case-management process with ownership, enrichment, escalation history, and post-incident review.

The fourth research question was about simulated data. The main weakness is not that the data is synthetic by itself, but that it is too clean. The samples are useful for proving the workflow and checking detector behaviour, but they cannot show how the system behaves with missing fields, inconsistent timestamps, normal operational noise, misconfigured hosts, real user behaviour, or mixed traffic from many devices. The Wazuh validations make the report stronger because selected alerts were handled by a real SIEM, but they do not replace later testing with a persistent endpoint or more realistic network telemetry.

10. Discussion

At this stage the project is best understood as a controlled engineering exercise rather than a small copy of a production SOC. It generates or reads a small input, applies a detector, preserves evidence, writes an incident note, and validates selected results through Wazuh. The endpoint-container syslog test is an important improvement over the earlier localfile tests because the SSH event no longer originates from a file inside the Wazuh manager. It is still a lab path, but it is closer to how a separate host would send evidence into a monitoring system.

The strongest part of the project is repeatability. A simulator scenario can be rerun with a fixed run ID and seed, and the sample-log detectors use short input files that can be inspected by hand. This matters because the result is not just a screenshot or a one-time demo; the evidence can be regenerated, tested, and compared. JSONL also works well here because it is readable during development and can later be forwarded, indexed, or transformed without changing the detector logic.

The weakest part is realism. The current inputs come from a Python simulator, hand-written sample logs, Docker-contained live files, or a short-lived endpoint container. They do not yet include a persistent Linux endpoint with a Wazuh agent, real firewall logs, DHCP or ARP observations, DNS or proxy telemetry, Modbus TCP messages, or MQTT traffic. The Wazuh tests and dashboard screenshots therefore show that selected ingestion paths work. They do not show broad monitoring coverage.

11. Limitations and Future Work

The main limitations are practical rather than theoretical. The project proves the workflow, but it does not yet prove that the same approach will behave well in a noisy environment.

- Telemetry is simulated or synthetic rather than collected from real industrial hardware.
- The eight detection categories are still narrow and mostly based on thresholds, allowlists, or exact sample formats.
- Timing is measured for local simulator and sample-log runs, not for live Wazuh ingestion from a persistent endpoint.
- The Wazuh validation covers three controlled paths: exported Mini-SOC JSON, a manager-local auth log, and an endpoint-container syslog event.
- The project does not yet include a Wazuh-agent host, firewall logs, DNS logs, DHCP/ARP observations, Modbus traffic, MQTT traffic, or a network sensor.

The most useful next technical step is still a persistent endpoint experiment, using either a small Linux VM or a Raspberry Pi with the Wazuh agent installed. That would make it possible to measure live

ingestion delay, endpoint event collection, and dashboard behaviour without replacing the Python detector work. If that setup has to wait, the project can still be published as a controlled prototype, as long as the report keeps the boundary clear.

Smaller improvements would also help before publication: more varied benign samples, a short demo recording, and a cleaner test layout if the repository keeps growing. The current test file is still manageable, but splitting it by detector would make later maintenance easier.

12. Conclusions

Industrial Mini-SOC has reached a useful controlled-prototype stage. It contains a working Python codebase, reproducible sample data, incident notes, measured local evaluations, automated tests, package metadata, CI checks, and Wazuh validation for selected paths. The implemented scenarios cover simulated process anomalies, suspicious command behaviour, invalid machine states, a synthetic failed-login burst, port-scan-like connection behaviour, unknown-device observations, and suspicious outbound connections.

The research questions can be answered within that scope. The prototype detects the selected event categories, behaves consistently against the designed test set, explains alerts in a readable way, and shows the limits of simulated data instead of hiding them. The results are meaningful because they are tied to files in the repository: source code, sample inputs, generated evidence, tests, incident notes, Wazuh evidence files, and dashboard screenshots. They are not evidence of production readiness, and the report should not present them that way.

The project is suitable for publication as a focused technical report and portfolio artefact. Its value is the engineering path: define a narrow threat model, implement the detectors, test benign and suspicious samples, preserve the evidence, validate selected SIEM paths, and state clearly what the system can and cannot show.